Entendendo o CSS
Guia comentado para alunos

Análise linha a linha de uma folha de estilos real:
variáveis, reset, formulários, botões e responsividade

Material de apoio • HTML & CSS

Sumário

1. Variáveis CSS (:root)
2. Reset de estilos
3. Estilo base do body
4. Acessibilidade com :focus-visible
5. Cabeçalho e rodapé agrupados
6. Formulário e campos de entrada
7. Transições (animações suaves)
8. Botão com efeito de "levantar"
9. Seletor :last-child
10. Media Query (responsividade)

Tabela-resumo de conceitos-chave

1 Variáveis CSS (:root)

```
:root {
--primary: #0d6efd;
--success: #198754;
...
}
```

O **:root** representa o elemento raiz do documento HTML (basicamente a tag mais alta, <html>). Dentro dele, criamos **variáveis CSS** (também chamadas de *custom properties*) usando a sintaxe **--nome-da-variavel**.

Para **usar** uma variável em qualquer lugar do CSS, escrevemos `var(--nome-da-variavel)`.

> **■ Por que usar variáveis?**
> Se quisermos trocar a cor azul do site inteiro, mudamos em um único lugar em vez de em dezenas de linhas espalhadas. Isso facilita manter consistência visual (mesma cor, mesma sombra, mesma borda em todos os elementos).

2 Reset de estilos

```
*, *::before, *::after {
margin: 0;
padding: 0;
box-sizing: border-box;
}
```

- O * seleciona **todos** os elementos da página.
- `::before` e `::after` são "pseudo-elementos" (conteúdo gerado via CSS, como ícones decorativos).

Zeramos **margin** e **padding** porque cada navegador aplica espaçamentos padrão diferentes — isso garante que todos comecem do zero, e nós controlamos o espaçamento manualmente.

> ■ **box-sizing: border-box**
>
> Faz com que padding e border sejam **incluídos** na largura/altura total do elemento, em vez de somados a ela. Isso evita muita dor de cabeça ao calcular tamanhos.

3 Estilo base do body

```
body {
font-family: 'Inter', 'Roboto', sans-serif;
background: var(--light);
color: var(--dark);
line-height: 1.6;
}
```

• **font-family** define uma lista de fontes por ordem de preferência: tenta 'Inter', se não tiver, tenta 'Roboto', senão usa qualquer fonte sans-serif do sistema.
• **line-height: 1.6** aumenta o espaçamento entre linhas de texto, melhorando a legibilidade.

4 Acessibilidade com :focus-visible

```
:focus-visible {
outline: 3px solid var(--primary);
outline-offset: 2px;
}
```

Quando alguém navega pelo site usando **Tab** no teclado (sem mouse), o navegador precisa mostrar visualmente qual elemento está selecionado. O `:focus-visible` desenha um contorno azul ao redor do elemento focado.

> **■ Por que isso importa?**
> Esse contorno é essencial para pessoas com deficiência visual ou motora que dependem do teclado para navegar pelo site. Sem ele, é impossível saber onde está o foco atual da página.

5 Cabeçalho e rodapé agrupados

```
header, footer {
background: var(--primary);
color: #fff;
text-align: center;
padding: 1rem;
}

header { font-size: 1.5rem; font-weight: 500; box-shadow: var(--shadow); }
footer { margin-top: 2rem; font-size: 0.9rem; }
```

Como `header` e `footer` compartilham vários estilos, agrupamos os seletores com **vírgula** para não repetir código. Depois, cada um recebe ajustes específicos separadamente, em regras individuais.

6 Formulário e campos de entrada

```
form {
max-width: 480px;
margin: 2rem auto;
...
}
```

• **max-width: 480px** impede que o formulário fique muito largo em telas grandes.
• **margin: 2rem auto** centraliza o formulário horizontalmente (o "auto" nas laterais faz esse truque de centralização).

```
input:focus {
border-color: var(--primary);
box-shadow: 0 0 0 0.25rem rgba(13,110,253,0.25);
outline: none;
}
```

Quando o usuário clica em um campo (`:focus`), a borda muda de cor e aparece um "halo" suave ao redor (simulado com box-shadow). Isso dá feedback visual claro de qual campo está ativo.

7 Transições (animações suaves)

```
transition: border-color var(--transition), box-shadow var(--transition);
```

A propriedade **transition** diz ao navegador: "quando essas propriedades mudarem de valor, não troque instantaneamente — faça a mudança de forma suave ao longo do tempo definido" (aqui, 0.3s). Isso evita mudanças bruscas e dá uma sensação mais profissional à interface.

8 Botão com efeito de "levantar"

```
button:hover {
background: var(--success-hover);
transform: translateY(-2px);
}
```

• **transform: translateY(-2px)** move o botão 2 pixels para cima quando o mouse passa por cima.
• Combinado com a **transition**, isso cria uma pequena animação de "flutuação", dando sensação de interatividade ao clicar ou passar o mouse.

9 Seletor :last-child

```
article:last-child { border-bottom: none; }
```

Isso seleciona o **último** elemento `article` dentro do seu container e remove a borda inferior — evita ter uma linha "sobrando" depois do último item da lista.

10 Media Query (responsividade)

```
@media (max-width: 600px) {
form, section, aside { margin: 1rem; padding: 1rem; }
}
```

@media permite aplicar regras CSS **somente** quando certa condição é satisfeita — aqui, quando a tela tem largura máxima de 600px (celulares). Assim, reduzimos as margens e paddings para aproveitar melhor o espaço pequeno da tela.

Conceitos-chave para reforçar com a turma

Conceito	Para que serve
Variáveis (--nome)	Reutilizar valores e facilitar manutenção do código
box-sizing: border-box	Simplificar o cálculo de tamanhos de elementos
:focus-visible	Acessibilidade para navegação via teclado
transition	Suavizar mudanças de estado (cores, sombras, posição)
:last-child, :hover	Pseudo-classes para estilizar por posição ou interação
@media	Responsividade — adaptar o layout a diferentes telas

■ Atividade sugerida

Peça aos alunos para trocarem os valores das variáveis em :root e observarem como toda a página muda automaticamente. Em seguida, peça para removerem a regra de reset (*) e comparar as diferenças de espaçamento entre navegadores.